```
  DB[:expenses].insert(expense)
  id = DB[:expenses].max(:id)
  RecordResult.new(true, id, nil)
end
```

When you rerun RSpec, your integration spec should pass now.

Testing the Invalid Case

So far, our integration spec checks only the "happy path" of saving a valid expense. Let's add a second spec that tries an expense missing a payee:

06-integration-specs/07/expense_tracker/spec/integration/app/ledger_spec.rb

```
context 'when the expense lacks a payee' do
  it 'rejects the expense as invalid' do
    expense.delete('payee')

    result = ledger.record(expense)

    expect(result).not_to be_success
    expect(result.expense_id).to eq(nil)
    expect(result.error_message).to include('`payee` is required')

    expect(DB[:expenses].count).to eq(0)
  end
end
```

Here, the Ledger instance should return the correct failure status and message, and the database should have no expenses in it.

Since this behavior isn't implemented, we want this new spec to fail:

```
$ bundle exec rspec spec/integration/app/ledger_spec.rb
« truncated »

Failures:

  1) ExpenseTracker::Ledger#record when the expense lacks a payee rejects ↩
  the expense as invalid

« truncated »

  2) ExpenseTracker::Ledger#record with a valid expense successfully saves ↩
  the expense in the DB

« truncated »

Finished in 0.02597 seconds (files took 0.18213 seconds to load)
2 examples, 2 failures

Failed examples:

rspec ./spec/integration/app/ledger_spec.rb:34 # ↩
ExpenseTracker::Ledger#record when the expense lacks a payee rejects the ↩
expense as invalid
```